AUDIO PROMPT ADAPTER: UNLEASHING MUSIC EDITING ABILITIES FOR TEXT-TO-MUSIC WITH LIGHTWEIGHT FINETUNING

Fang-Duo Tsai[1] **Shih-Lun Wu**[2] **Haven Kim**[3]
Bo-Yu Chen[1] **Hao-Chung Cheng**[1] **Yi-Hsuan Yang**[1]
[1] National Taiwan University [2] Carnegie Mellon University [3] University of California San Diego
r12942150@ntu.edu.tw, shihlunw@andrew.cmu.edu, khaven@ucsd.edu
bernie40916@gmail.com, haochung@ntu.edu.tw, yhyangtw@ntu.edu.tw

ABSTRACT

Text-to-music models allow users to generate nearly realistic musical audio with textual commands. However, *editing* music audios remains challenging due to the conflicting desiderata of performing fine-grained alterations on the audio while maintaining a simple user interface. To address this challenge, we propose *Audio Prompt Adapter* (or AP-Adapter), a lightweight addition to pretrained text-to-music models. We utilize AudioMAE to extract features from the input audio, and construct attention-based adapters to feed these features into the internal layers of AudioLDM2, a diffusion-based text-to-music model. With 22M trainable parameters, AP-Adapter empowers users to harness both global (e.g., genre and timbre) and local (e.g., melody) aspects of music, using the original audio and a short text as inputs. Through objective and subjective studies, we evaluate AP-Adapter on three tasks: timbre transfer, genre transfer, and accompaniment generation. Additionally, we demonstrate its effectiveness on out-of-domain audios containing unseen instruments during training.

1. INTRODUCTION

Advancements in *text-to-music generation* have made it possible for users to create music audio signals from simple textual descriptions [1–4]. To improve the control over the generated music beyond textual input, several newer models have been proposed, using additional conditioning signals indicating the intended global or time-varying musical attributes such as melody, chord progression, rhythm, or loudness for generation [5–9] (see Section 2 for a brief review). Such controllability is important for musicians, practitioners, as well as common users in the human-AI co-creation process [10, 11].

However, one area that remains challenging, which we refer to as *text-to-music editing* below, is the precise editing of a piece of music, provided by a user as an *audio input* $\boldsymbol{x}$ alongside the *text input* $\boldsymbol{y}$ for the textual prompts. The goal here for the model is to create an "edited" version of the input music, denoted as $\tilde{\boldsymbol{x}}$, according to the text input. This is a crucial capability for users who wish to refine either an original or machine-generated music without compromising its musicality and audio quality, while keeping the simplicity of text-based human-computer interaction. Namely, the desired properties of the output $\tilde{\boldsymbol{x}}$ are:

- **Transferability**: $\tilde{\boldsymbol{x}}$ should reflect what $\boldsymbol{y}$ specifies, e.g., timbre, genre, instrumentation, or mood.
- **Fidelity**: $\tilde{\boldsymbol{x}}$ should retain all other musical content in $\boldsymbol{x}$ that $\boldsymbol{y}$ does not concern, e.g., melody and rhythm.

While a text-to-music generation model takes in general only the text input $\boldsymbol{y}$ and generates music freely, a text-to-music editing model takes both audio and text inputs $\boldsymbol{x}$ and $\boldsymbol{y}$. The primary challenge arises from the conflicting goals of maintaining high fidelity to the input audio $\boldsymbol{x}$ while incorporating specific changes dictated by textual commands $\boldsymbol{y}$. As we review in Section 2, existing methods [14–16] either lack the granularity needed for detailed audio manipulation or need complex prompt engineering that detracts from user accessibility or requires iterative refinements.

A secondary challenge arises from the large number of trainable parameters needed for models to achieve high musical quality and diversity (e.g., MusicGen-medium [5] has 1.5B parameters). Without much computational resource, it is more feasible to treat existing models as "foundation models" and finetune them to fulfill specific needs, instead of training a model from scratch [17].

In view of these challenges, we propose in this paper the *Audio Prompt Adapter* (or, AP-Adapter for short), a novel approach inspired by the Image Prompt Adapter (IP-Adapter) [18] from the neighboring field of text-to-image editing. This lightweight (22M parameters), attention-based module integrates seamlessly with existing text-to-music generation models, specifically leveraging the pre-trained AudioLDM2 model [12] enhanced by the AudioMAE encoder [13] to extract audio features. Our method uniquely combines text and audio inputs through decoupled cross-attention layers, allowing precise control in the generation process. After training the AP-adapter with a single NVIDIA RTX 3090, our method can zero-shot edit a given

© F.Tsai, S.Wu, H.Kim, B.Chen, H.Cheng, Y.Yang. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** F.Tsai, S.Wu, H.Kim, B.Chen, H.Cheng, Y.Yang, "Audio Prompt Adapter: Unleashing Music Editing Abilities for Text-to-Music with Lightweight Finetuning ", in *Proc. of the 25th Int. Society for Music Information Retrieval Conf.,* San Francisco, United States, 2024.

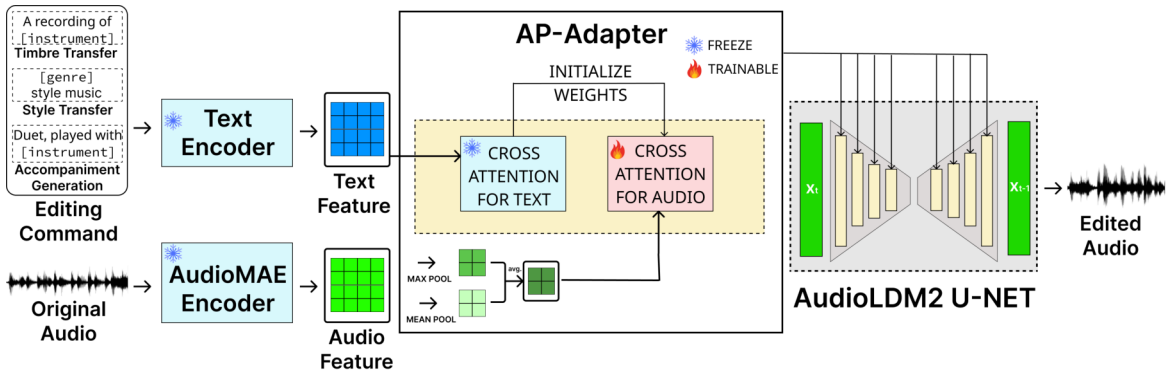


Figure 1: Our AP-Adapter is an add-on to AudioLDM2 [12]. Users provide an original audio to AudioMAE [13] to extract audio features, and an editing command to the text encoder. The decoupled audio and text cross-attention layers of AP-Adapter contribute to the **fidelity** with the input audio and **transferability** of the editing command in the edited audio.

audio prompt according to the text prompt.

Our AP-Adapter offers great improvements over some baseline models by enabling detailed and context-sensitive audio manipulations, achieving a balance between fidelity and the transferability effects dictated by user inputs. Our experiments across timbre transfer, genre transfer, and accompaniment generation tasks demonstrate the effectiveness of our approach in handling diverse and complex editing requirements. In short, our key contributions are:

- Proposing a framework that equips an audio input modality for a pre-trained text-to-music generation model.
- Performing zero-shot music editing with a lightweight adapter, which permits flexible balance of the effects of the text and audio inputs.
- Demonstrating three tasks: timbre transfer, genre transfer, accompaniment generation, and discussing the impact of tunable hyperparameters.

We provide audio examples in our demo website. [1] We also share source code and model checkpoints on GitHub. [2]

2. RELATED WORK

Generating desired music from text prompts alone is complex and often requires intricate prompt engineering. Mustango [7] enhanced prompts with information-rich captions specifying chords, beats, tempo, and key. MusicGen [5] conditioned music generation on melodies by extracting chroma features [19] and inputting them with the text prompt into a Transformer model. Coco-Mulla [6] and MusiConGen [9] extended MusicGen by adding time-varying chord- and rhythm-related controls. Music ControlNet [8] incorporated time-varying conditions like melody, rhythm, and dynamics for diffusion-based text-to-music models. These methods utilize low-level features to guide generation but do not take reference audio as input, limiting their potential for editing existing audio tracks.

Recently, several music editing methods were proposed. InstructME [14] uses a VAE and a chord-conditioned diffusion model for music editing but requires a large dataset of audio files with multiple instrumental tracks and triplet data of text instructions, source music, and target music for supervised training. M^{2}UGen [15] leverages large language models to understand and generate music across different modalities, supporting music editing via natural language, but it requires a three-step training process and complex preprocessing. MusicMagus [16] implements latent space manipulation during inference for music editing but requires an additional music captioning model and the InstructGPT LLM to address discrepancies between the text prompt distribution of AudioLDM2 and the music captioning model.

Compared to these methods, our AP-Adapter is more straightforward to train and can achieve multiple music editing tasks in a zero-shot manner.

3. BACKGROUND

3.1 Diffusion Model

Denoising diffusion probabilistic models (DDPMs) [20], also known as diffusion models, are a class of generative models that approximates some distribution $p(\boldsymbol{x})$ via denoising through a sequence of $T-1$ latent variables:

$$p_\theta(\boldsymbol{x}) = \int \Big[\prod_{t=1}^{T} p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)\Big] p(\boldsymbol{x}_T) d\boldsymbol{x}_{1:T}\,, \quad (1)$$

where θ is the set of learnable parameters, $\boldsymbol{x}_0 := \boldsymbol{x}$, and $p(\boldsymbol{x}_T) := \mathcal{N}(0, \mathbf{I})$ (i.e., an uninformative Gaussian prior). To train the model, we run forward diffusion: sample some data point $\boldsymbol{x} \sim p(\boldsymbol{x})$ and some $t \in [1, T]$, and add noise $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$ to $\boldsymbol{x}$ to produce a noised data point $\boldsymbol{x}_t := \sqrt{\bar{\beta}_t}\boldsymbol{x} + \sqrt{1-\bar{\beta}_t}\boldsymbol{\epsilon}$, where $\bar{\beta}_t$ is the pre-defined noise level for step t. The model is asked to perform backward diffusion, namely, to recover the added noise via the objective $\min_\theta \mathbb{E}_{\boldsymbol{x},\boldsymbol{\epsilon},t}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)\|_2^2\right]$, where $\boldsymbol{\epsilon}_\theta(\cdot)$ is the model's prediction, that is equivalent to maximizing the evidence lower bound (ELBO) of $p_\theta(\boldsymbol{x})$. During inference, we start from an $\boldsymbol{x}_T \sim \mathcal{N}(0, \mathbf{I})$ and iteratively remove the predicted noise $\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)$ to generate data. Song *et al.* [21] offered a crucial interpretation that each denoising step can be seen as ascending along $\nabla_{\boldsymbol{x}} \log p_\theta(\boldsymbol{x})$, also known as the *score* of $p_\theta(\boldsymbol{x})$. Any input condition $\boldsymbol{y}$ can be incorporated into a

[1] Demo: https://rebrand.ly/AP-adapter
[2] Code: https://github.com/fundwotsai2001/AP-adapter

diffusion model by injecting embeddings of $\boldsymbol{y}$ via, for example, cross-attention [22], thereby modeling $p_\theta(\boldsymbol{x} \mid \boldsymbol{y})$ (and $\nabla_{\boldsymbol{x}} \log p_\theta(\boldsymbol{x} \mid \boldsymbol{y})$). To reduce memory footprint and accelerate training/inference, latent diffusion models (LDMs) [22] proposed to first compress data points $\boldsymbol{x}$ into latent vectors using a variational autoencoder (VAE) [23], and then learn a diffusion model for the latent vectors.

3.2 AudioLDM2

We choose AudioLDM2 [12], a latent diffusion-based [22] text-to-audio model, as our pretrained backbone. To enable text control over generated audio, AudioLDM2 uses AudioMAE [13] to extract acoustic features, named the *language of audio* (LOA), from the target audio. LOA serves as the bridge between acoustic and text-centric semantic information—the text prompt is encoded by both the FLAN-T5 [24] language model and CLAP [25] text encoder (which has a joint audio-text embedding space), and then passed to a trainable GPT-2 [26] to approximate the LOA via a regression loss that aligns the semantic representations with LOA. The aligned text information is then fed into the U-Net [27] for diffusion process to influence the generation. We pick AudioLDM2 to be the backbone since the use of LOA likely promotes the affinity to accepting audio conditions, which is crucial to our fidelity goal.

3.3 Classifier-free Guidance

Classifier-free guidance (CFG) [28] is a simple yet effective inference-time method to enhance the input text condition's influence, which is directly linked to our transferability goal. As mentioned in Sec. 3.1, diffusion models can predict both the unconditioned score $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$ and the conditioned score $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x} \mid \boldsymbol{y})$. In addition, by Bayes' rule, we know that $p(\boldsymbol{x} \mid \boldsymbol{y}) \propto p(\boldsymbol{x})p(\boldsymbol{y} \mid \boldsymbol{x})$. As the goal is the amplify $\boldsymbol{y}$'s influence, we define:

$$p_\lambda(\boldsymbol{x} \mid \boldsymbol{y}) :\propto p(\boldsymbol{x})p(\boldsymbol{y} \mid \boldsymbol{x})^\lambda \,, \tag{2}$$

where λ is a knob, named *CFG scale*, that controls the strength of $\boldsymbol{y}$. Taking $(\nabla_{\boldsymbol{x}} \log)$ on both sides gives us:

$$\nabla_{\boldsymbol{x}} \log p_\lambda(\boldsymbol{x} \mid \boldsymbol{y}) = \lambda \nabla_{\boldsymbol{x}} \log p(\boldsymbol{y} \mid \boldsymbol{x}) + \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) \,. \tag{3}$$

Meanwhile, we can rearrange the Bayes' rule terms to get:

$$\nabla_{\boldsymbol{x}} \log p(\boldsymbol{y} \mid \boldsymbol{x}) = \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x} \mid \boldsymbol{y}) - \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) \,. \tag{4}$$

Note that a diffusion model can predict both RHS terms. Plugging Eqn. (4) into Eqn. (3), CFG performs

$$\begin{aligned} \nabla_{\boldsymbol{x}} \log p_\lambda(\boldsymbol{x} \mid \boldsymbol{y}) &= \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) \\ &\quad + \lambda(\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x} \mid \boldsymbol{y}) - \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})) \end{aligned} \tag{5}$$

at every inference iteration, where $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$ is obtained by inputting an empty string as $\boldsymbol{y}$.

4. PROPOSED AUDIO PROMPT ADAPTER

To effectively condition AudioLDM2 on the input audio and achieve our transferability and fidelity goals, our AP-Adapter adds two components to AudioLDM2: an audio encoder to extract acoustic features, and decoupled cross-attention adapters to incorporate the acoustic features while maintaining text conditioning capability.

4.1 Audio Encoder and Feature Pooling

We adopt AudioMAE as the audio encoder, which is used by AudioLDM2 to produce the language of audio (LOA; see Section 3.2) during its training. In our pilot study, we find that using the LOA directly as the condition causes nearly verbatim reconstruction, i.e., information in the input audio is mostly retained. This is undesirable as it greatly limits transferability. To address this issue, we apply a combination of max and mean pooling on the LOA, and leave the pooling rate, which we denote by ω, tunable by the user to trade off between fidelity and transferability.

4.2 Decoupled Cross-attention Adapters

According to the analyses in [29, 30] performed on text-to-image diffusion models finetuned for image editing [31], the cross-attention layers, which allow interaction between text prompt and the diffusion process, undergo the most drastic changes during fine-tuning. Hence, we implement our AP-Adapter also as a set of cross-attention layers.

Recall that the audio and text prompts are transformed to internal features before interacting with the U-Net for diffusion. We define these features as:

$$\boldsymbol{c}_{\boldsymbol{x}} := \text{Pool}(\text{AudioMAE}(\boldsymbol{x})) \tag{6}$$

$$\boldsymbol{c}_{\boldsymbol{y}} := \text{GPT2}([\text{FlanT5}(\boldsymbol{y}); \text{CLAP}(\boldsymbol{y})]) \,, \tag{7}$$

where $\boldsymbol{c}_{\boldsymbol{x}}$ and $\boldsymbol{c}_{\boldsymbol{y}}$ are the audio and text features respectively. The original AudioLDM2 incorporates the text feature into each U-Net layer via cross-attention:

$$\boldsymbol{z}_{\text{text}} := \text{Attention}(\boldsymbol{z}\boldsymbol{W}^{(q)}, \boldsymbol{c}_{\boldsymbol{y}}\boldsymbol{W}^{(k)}, \boldsymbol{c}_{\boldsymbol{y}}\boldsymbol{W}^{(v)}) \,, \tag{8}$$

where $\boldsymbol{z}$ is the U-Net's internal feature, and $\boldsymbol{W}^{(q)}$, $\boldsymbol{W}^{(k)}$, $\boldsymbol{W}^{(v)}$ are learnable projections that respectively produce the cross-attention query, key, and values from $\boldsymbol{z}$ or $\boldsymbol{c}_{\boldsymbol{y}}$. We keep this cross-attention for text intact (i.e., frozen), anticipating it to satisfy transferability out of the box.

To incorporate the audio features for fidelity, we place a decoupled audio cross-attention layer as the adapter alongside each text cross-attention in a similar light to [18]:

$$\boldsymbol{z}_{\text{audio}} := \text{Attention}(\boldsymbol{z}\boldsymbol{W}^{(q)}, \boldsymbol{c}_{\boldsymbol{x}}\boldsymbol{W}'^{(k)}, \boldsymbol{c}_{\boldsymbol{x}}\boldsymbol{W}'^{(v)}) \,, \tag{9}$$

where $\boldsymbol{W}'^{(k)}$ and $\boldsymbol{W}'^{(v)}$ are the newly introduced adapter weights. Since during AudioLDM2 training, the text feature $\boldsymbol{c}_{\boldsymbol{y}}$ is trained to mimic the LOA from AudioMAE, we initialize $\boldsymbol{W}'^{(k)}$ and $\boldsymbol{W}'^{(v)}$ respectively from $\boldsymbol{W}^{(k)}$ and $\boldsymbol{W}^{(v)}$ for all the cross-attention layers in the Unet, and find that this significantly shortens our fine-tuning process compared to random initialization.

Finally, we obtain the final output of the decoupled text and audio cross-attentions via a weighted sum:

$$\boldsymbol{z}_{\text{fusion}} := \boldsymbol{z}_{\text{text}} + \alpha \boldsymbol{z}_{\text{audio}} \,, \tag{10}$$

where $\alpha \in \mathbb{R}$, named *AP scale*, is a hyperparameter that controls the strength of the audio prompt (fixed to $\alpha = 1$ during training), and $\boldsymbol{z}_{\text{fusion}}$ becomes the input of the subsequent U-Net layer. We expect $\boldsymbol{z}_{\text{fusion}}$ to capture the information mixture from audio and text prompts, inducing the model to generate plausible music that adheres to both.

4.3 Training

We freeze all the parameters in the pretrained AudioLDM2 and AudioMAE, except for the decoupled audio cross-attention adapters with 22M parameters. The loss function follows that of standard (latent) diffusion models:

$$\mathcal{L} = \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y}),\boldsymbol{\epsilon},t} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta \left(\boldsymbol{x}_t, \boldsymbol{c}_{\boldsymbol{x}}, \boldsymbol{c}_{\boldsymbol{y}}, t \right) \right\|_2^2 , \qquad (11)$$

where $(\boldsymbol{x}, \boldsymbol{y})$ are naturally existing paired audio and text, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$, t is the diffusion step, $\boldsymbol{x}_t$ is the noised audio latent features, $\boldsymbol{c}_{\boldsymbol{x}}, \boldsymbol{c}_{\boldsymbol{y}}$ are the extracted features from text and audio prompts (cf. Eqn. (6) and (7)), and $\boldsymbol{\epsilon}_\theta(\cdot)$ is the model's predicted noise. Minimizing $\mathcal{L}$ is equivalent to maximizing the lower bound of $p(\boldsymbol{x} \mid \boldsymbol{c}_{\boldsymbol{x}}, \boldsymbol{c}_{\boldsymbol{y}})$. During training, we select the audio feature's pooling rate ω from the set $\{1, 2, 4, 8\}$ uniformly at random, making the adapters recognize audio features with different resolutions, thereby allowing users to balance fidelity and transferability at inference. Additionally, we randomly dropout audio and text conditions, i.e., setting $\boldsymbol{c}_{\boldsymbol{x}}$ to a zero matrix, and $\boldsymbol{y}$ to an empty string, to facilitate classifier-free guidance.

4.4 Inference

At inference, users are free to input any text prompt $\boldsymbol{y}$ as the editing command to achieve their desired edits, i.e, $\boldsymbol{x} \to \tilde{\boldsymbol{x}}$. In addition, following [32,33], we modify the unconditioned terms in Eqn. (5) using a negative text prompt $\boldsymbol{y}^-$. Letting $\boldsymbol{c}_{\boldsymbol{xy}} := \{\boldsymbol{c}_{\boldsymbol{x}}, \boldsymbol{c}_{\boldsymbol{y}}\}$, our inference step is:

$$\begin{aligned} \nabla_{\tilde{\boldsymbol{x}}} \log p_\lambda(\tilde{\boldsymbol{x}} \mid \boldsymbol{c}_{\boldsymbol{xy}}, \boldsymbol{c}_{\boldsymbol{y}^-}) &= \nabla_{\tilde{\boldsymbol{x}}} \log p(\tilde{\boldsymbol{x}} \mid \boldsymbol{c}_{\boldsymbol{y}^-}) \\ &+ \lambda \left(\nabla_{\tilde{\boldsymbol{x}}} \log p(\tilde{\boldsymbol{x}} \mid \boldsymbol{c}_{\boldsymbol{xy}}) - \nabla_{\tilde{\boldsymbol{x}}} \log p(\tilde{\boldsymbol{x}} \mid \boldsymbol{c}_{\boldsymbol{y}^-}) \right) \quad (12) \end{aligned}$$

We find that specifying $\boldsymbol{y}^-$ is an effective way to avoid unwanted properties in $\tilde{\boldsymbol{x}}$, e.g., the original timbre for the timbre transfer task, or low-quality music in general.

5. EXPERIMENT SETUP

5.1 Dataset Preparation

For the training data of our AP-Adapter, due to our limited computation resource, we use 200K 10-second-long audios with text tags randomly sampled from AudioSet [34] (about 500 hours, or ∼10% of the whole dataset).

For the audio input $\boldsymbol{x}$ used in evaluation, we compile two datasets: **in-domain** and **out-of-domain**, according to whether the AudioSet ontology includes the instrument.

- **In-domain**: We choose 8 common instruments: `piano`, `violin`, `cello`, `flute`, `marimba`, `organ`, `harp` and `acoustic guitar`. For each instrument, we manually download 5 high-quality monophonic audios from YouTube (i.e., 40 samples in total) and crop them each to 10 seconds.
- **Out-of-domain**: We collect a dataset of monophonic melodies played by ethnic instruments, including 2 *Chinese* instruments (collected by one of our co-authors) and 5 *Korean* instruments (downloaded from AIHub [35]). We use 5 audio samples for each instrument (35 audios in total), cropped to 10 seconds each. We note that these instruments are **not seen** during the training time.

Except for the Korean data which is not licensed outside of Korea, we share information to get the data on GitHub.

5.2 Evaluation Tasks

By varying the edit command $\boldsymbol{y}$, we evaluate AP-Adapter on three music editing tasks:

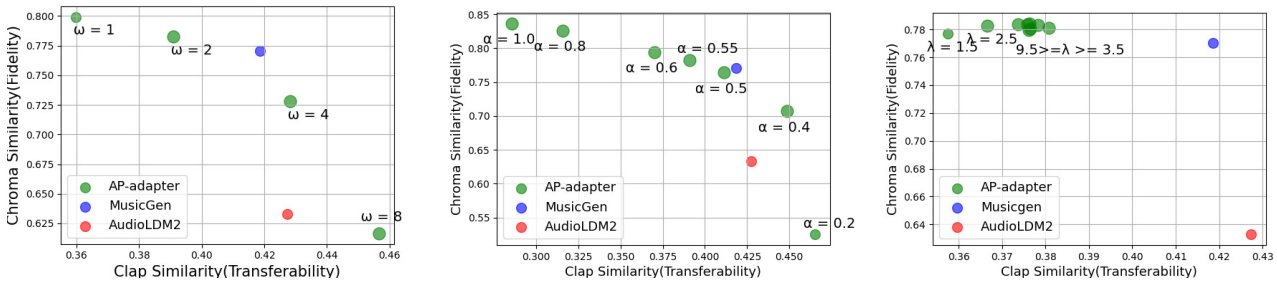
- **Timbre transfer**: The model is expected to change a melody's timbre to that of the target instrument, and keep all other contents unchanged. For this task, the editing command ($\boldsymbol{y}$) is set to "a recording of a `[target instrument]` solo". The negative prompt ($\boldsymbol{y}^-$) is "a recording of the `[original instrument]` solo". For in-domain input, the target is one of the other 7 in-domain instruments. For out-of-domain input, the target is one of the 8 in-domain instruments. We only use in-domain instruments as the target because our evaluation metrics CLAP [25] and FAD [36] (see Section 5.5) do not recognize the out-of-domain instruments.
- **Genre transfer**: We expect the genre (e.g., jazz and country) to change according to the text prompt, but we wish to retain most of the other content such as melody, rhythm and timbre. Here, we set $\boldsymbol{y}$:= "`[target genre]` style music", and $\boldsymbol{y}^-$:= "low quality music". Here, we target 8 genres: `jazz`, `reggae`, `rock`, `metal`, `pop`, `hip-hop`, `disco`, `country`.
- **Accompaniment generation**: We expect that all content in the input melody remains unchanged, but a new instrument is added to accompany the original audio in a pleasant-sounding and harmonic way. We set $\boldsymbol{y}$:= "Duet, played with `[accomp instrument]` accompaniment", and $\boldsymbol{y}^-$:= "low quality music". The [accomp instrument] is selected in the same way as the [target instrument] in the timbre transfer task.

We include these representative tasks which musicians may find useful for their daily workflow, but since $\boldsymbol{y}$ is free-form text, AP-Adapter has the potential for many other tasks.

5.3 Training and Inference Specifics

We use AudioLDM2-large (1.5B parameters), available on HuggingFace, as our backbone model, and only train our 22M-parameter adapters. Training is done on a single one RTX 3090 (24GB) for 35K steps with an effective batch size of 32. We use AdamW optimizer with fixed learning rate 10^{-4} and weight decay 10^{-2}. To enable CFG, we randomly dropout text and audio features with a 5% probability.

For inference, we choose the critical hyperparameters, i.e., pooling rate ω, AP scale α, and CFG scale λ, by exploring the transferability-fidelity tradeoff space as will be reported in Section 6.1. For timbre transfer and accompaniment generation, we select $\omega = 2$, $\alpha = 0.5$, $\lambda = 7.5$. For the



(a) Tuning pooling rate ω (b) Tuning AP scale α (c) Tuning classifier-free guidance scale λ

Figure 2: Transferability-fidelity tradeoff effects of different hyperparameters on the timbre transfer task. The hyperparameters are set to $\omega = 2$, $\alpha = 0.55$, and $\lambda = 7.5$ when they are not the hyperparameter of interest.

genre transfer , we select $\omega = 1$, $\alpha = 0.4$, $\lambda = 7.5$. Following AudioLDM2, we use 50 diffusion steps.

5.4 Baselines

We choose two well-known and publicly-available audio generation models, AudioLDM2 [12] and MusicGen [5], as our baselines. Both of them can generate nearly realistic music. We describe below how we use them for editing:

- **AudioLDM2**: Following SDEdit [37], we perform the forward process (i.e., adding noise to the audio input $\boldsymbol{x}$) partially for $0.75T$ steps, where T is the original number diffusion steps, and then denoise it back with the editing command $\boldsymbol{y}$ to obtain $\tilde{\boldsymbol{x}}$.
- **MusicGen**: MusicGen is a Transformer-based text-to-audio model that generates discrete audio tokens. We use MusicGen-Melody (1.5B), which achieves melody conditioning using chromagram [19] as a proxy. We input $\boldsymbol{y}$ as the text prompt, and the chromagram of $\boldsymbol{x}$ as the audio condition, for MusicGen to generate $\tilde{\boldsymbol{x}}$.

We do not include the recent text-to-music editing methods InstructME [14] or MusicMagus [16], as they have not released the code and models, and also exclude M^{2}UGen [15] as it is heavily focused on music understanding and visually-conditioned music generation.

5.5 Objective Metrics

We employ the following metrics:

- **CLAP** [25] is used to evaluate **transferability**, as it is trained with contrastive losses to align the representations for audio and text. We compute the cosine similarity between CLAP audio embedding for the edited audio $\tilde{\boldsymbol{x}}$ and CLAP text embedding for the command $\boldsymbol{y}$.[3] Higher scores show high semantic relevance between $\tilde{\boldsymbol{x}}$ and $\boldsymbol{y}$.
- **Chroma similarity** computes the similarity of the original and edited audios $\tilde{\boldsymbol{x}}$ and $\boldsymbol{x}$ harmonically and rhythmically, thereby evaluates **fidelity**. We adopt librosa's [38] CQT chroma method to extract the 12-dimensional chromagrams [19] to compute framewise cosine similarity.

[3] For accompaniment generation task, text input to CLAP is modified to include both instruments, e.g., "Piano duet, played with violin."

Model	CLAP↑ (transferability)	Chroma↑ (fidelity)	FAD↓ (overall)
MusicGen	**0.339**	0.771	8.443
AudioLDM2	0.284	0.643	**5.389**
AP-Adapter	0.314	**0.777**	5.986

Table 1: Objective evaluation on *in-domain* audio inputs of MusicGen-Melody [5], AudioLDM2-SDEdit [12, 37], and the proposed AP-Adapter. Results are the average of the three tasks. Best results are highlighted in bold (↑ / ↓: the higher / lower the better).

- **Fréchet audio distance (FAD)** [36] uses a pretrained audio classifier to extract audio features, collects features from all audios, and estimates the feature covariance matrix. Then, the Fréchet distance is computed between the two covariance matrices (one from generated audios, one from real audios). We adopt FAD to evaluate the **overall quality/realisticness** of the generations. Following the official implementation, we use VGGish architecture [39] as the feature extractor. We use the in-domain evaluation dataset as real audios.

5.6 Subjective Study

We design a listening test that contains 2 sets of music for each of the three tasks. The sets are independent from one another, and each contains a 10-second original audio prompt $\boldsymbol{x}$, an editing text command $\boldsymbol{y}$, and three edited audios $\tilde{\boldsymbol{x}}$ generated by our model and the two baselines (with order randomized and kept secret to participants). Participants rate each edited audio on a 5-point Likert scale, according to the following 3 aspects:

- **Transferability**: Do you feel that the generated audio matches what the text prompt asks for?
- **Fidelity**: Do you feel that the generated audio faithfully keeps the original musical content that should not be changed by the text prompt?
- **Overall preference**: Overall, how much do you like the generated audio?

We recruit 30 participants from our social circle and randomly assign them one of the 6 test suites (3 for in-domain, 3 for out-of-domain). The study takes about 10 minutes.

	Metric	**Transferability MOS**			**Fidelity MOS**			**Overall MOS**		
Eval. audios	*Task*	Timbre	Genre	Accomp.	Timbre	Genre	Accomp.	Timbre	Genre	Accomp.
In-domain	MusicGen	3.35	3.15	3.32	2.62	2.85	2.76	3.06	3.03	2.91
	AudioLDM2	3.21	2.74	3.12	2.21	2.21	2.26	2.47	2.56	2.47
	AP-Adapter	**3.59**	**3.44**	**3.41**	**3.47**	**3.74**	**3.41**	**3.26**	**3.44**	**3.12**
Out-of-domain	MusicGen	**2.92**	**3.96**	3.00	2.73	3.31	2.54	2.58	**3.58**	2.65
	AudioLDM2	2.62	2.12	2.96	2.42	2.69	2.23	2.58	2.31	2.81
	AP-Adapter	**2.92**	3.19	**3.54**	**3.81**	**3.58**	**3.96**	**3.08**	3.12	**3.31**

Table 2: Subjective study results (mean opinion scores $\in [1, 5]$) with 17 and 13 participants for in-domain and out-of-domain input audios, respectively, for the three evaluation tasks: timbre transfer, genre transfer, and accompaniment generation.

6. RESULTS AND DISCUSSION

6.1 Hyperparameter Choices

We discover in our early experiments that several hyperparameters, which are tunable during inference, can drastically affect the edited outputs. Therefore, we conduct a systematic study on the effects of audio pooling rate ω (Sec. 4.1), AP scale in decoupled cross-attention α (Sec. 4.2), and classifier-free guidance scale λ (Sec. 3.3). Specifically, we observe how their various values induce different behaviors on the transferability-fidelity plane spanned by CLAP and chroma similarity metrics.

- The **pooling rate** ω controls the amount of information from the audio prompt. Figure 2a shows clearly that when the pooling rate is low, the fidelity is higher, but at the cost of transferability. For example, the audio generated with $\omega = 1$ preserves abundant acoustic information, thus the edited audio sounds like the input audio, but it might not reflect the editing command. The opposite can be said for $\omega = 8$. Overall, $\omega = 2$ or 4 strikes a good balance.
- The **AP scale** α adjusts the relative importance between the text and audio decoupled cross-attentions. As opposed to pooling rate, it enhances fidelity at the expense of transferability at higher values, as shown in Figure 2b, and $\alpha \in [0.4, 0.6]$ leads to a more balanced performance.
- The **CFG guidance scale** λ dictates the strength of text condition as detailed in Eqn. (5). As shown in Figure 2c, somewhat unexpectedly, λ does not impact the tradeoff too much when $\lambda \geq 3.5$. Hence, we use $\lambda = 7.5$ across all tasks following AudioLDM2.

6.2 Objective Evaluations

We show the metrics computed on in-domain audios in Table 1, taking the average across the three editing tasks. (We do not report the result for out-of-domain audio inputs as we expect CLAP and FAD to be less reliable there.) In general, AP-Adapter exhibits the most well-rounded performance without significant weaknesses—MusicGen scores high on transferability, but has a much worse FAD score, indicating issues on quality or distributional deviation. We infer that, since MusicGen only considers melody as input rather than the entire audio, it has fewer limitations in the generating process and thus achieves a higher transferability score. On the other hand, AudioLDM2 consistently achieves the best FAD score but lacks fidelity and transferability.

We also evaluate the ablated version of AP-Adapter without using the negative prompt ($\boldsymbol{y}^-$). For the timber transfer task, not using the negative prompt induces worse transferability, degrading the CLAP score from 0.405 to 0.378, but does not negatively impact chroma similarity and FAD.

6.3 Subjective Evaluations

Table 2 shows the results from our listening test. Our AP-adapter outperforms the two other baseline models in 16 out of 18 comparisons. On top of preserving fine-grained details in the input audio, AP-adapter also tightly follows the editing commands and generate relatively high-quality music, leading in transferability and overall preference except for only the genre transfer task on out-of-domain audios. MusicGen performs better in transferability for genre transfer, but its fidelity is weaker as it only considers the melody of the input audio. With the additional audio-modality condition, AP-adapter has the advantage of "listening" to all the details of the input audio, receiving significantly higher scores on fidelity on both in- and out-of-domain cases.

The advantage of AP-adapter in fidelity is much stronger in Table 2 rahter than in Table 1. We conjecture that chroma similarity paints only a partial picture for fidelity as it is focused primarily on harmonic properties, leaving out other musical elements such as dynamics and percussive patterns.

7. CONCLUSIONS

We presented AP-Adapter, a lightweight add-on to AudioLDM2 that empowers it for music editing. AP-Adapter leverages AudioMAE to extract fine-grained features from the audio prompt, and feeds such features into AudioLDM2 via decoupled cross-attention adapters for effective conditioning. With only 500 hours of training data and 22M trainable parameters, AP-Adapter delivers compelling performance across useful editing tasks, namely, timbre transfer, genre transfer, and accompaniment generation. Additionally, it enables users to manipulate the transferability-fidelity tradeoff, and edit out-of-domain audios, which promotes creative endeavors with ethnic instrument audios that are usually scarce in publicly available datasets.

Promising directions for follow-up works include: *(i)* exploring more diverse editing tasks under our framework with various editing commands, *(ii)* extending AP-Adapter to other generative backbones, e.g., autoregressive models, and *(iii)* adding support for localized edits that can be stitched seamlessly with unchanged audio segments.

8. ACKNOWLEDGMENT

The work is supported by grants from the National Science and Technology Council of Taiwan (NSTC 112-2222-E-002-005-MY2) and (NSTC 113-2628-E-002-029), and the Ministry of Education (NTU-112V1904-5).

9. REFERENCES

[1] S. Forsgren and H. Martiros, "Riffusion: Stable diffusion for real-time music generation," 2022. [Online]. Available: https://riffusion.com

[2] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi *et al.*, "MusicLM: Generating music from text," *arXiv preprint arXiv:2301.11325*, 2023.

[3] H. Liu, Z. Chen, Y. Yuan, X. Mei, X. Liu, D. Mandic, W. Wang, and M. D. Plumbley, "AudioLDM: Text-to-audio generation with latent diffusion models," in *Proceedings of International Conference on Machine Learning (ICML)*, 2023.

[4] Q. Huang, D. S. Park, T. Wang, T. I. Denk, A. Ly, N. Chen, Z. Zhang, Z. Zhang, J. Yu, C. Frank, J. Engel, Q. V. Le, W. Chan, Z. Chen, and W. Han, "Noise2music: Text-conditioned music generation with diffusion models," *arXiv preprint arXiv:2302.03917*, 2023.

[5] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and controllable music generation," *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[6] L. Lin, G. Xia, J. Jiang, and Y. Zhang, "Content-based controls for music large language modeling," *arXiv preprint arXiv:2310.17162*, 2023.

[7] J. Melechovsky, Z. Guo, D. Ghosal, N. Majumder, D. Herremans, and S. Poria, "Mustango: Toward controllable text-to-music generation," in *Proceedings of Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

[8] S.-L. Wu, C. Donahue, S. Watanabe, and N. J. Bryan, "Music ControlNet: Multiple time-varying controls for music generation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2692–2703, 2024.

[9] Y.-H. Lan, W.-Y. Hsiao, H.-C. Cheng, and Y.-H. Yang, "MusiConGen: Rhythm and chord control for Transformer-based text-to-music generation," in *International Society for Music Information Retrieval Conference (ISMIR)*, 2024.

[10] C. A. Huang, H. V. Koops, E. Newton-Rex, M. Dinculescu, and C. J. Cai, "Human-AI co-creation in songwriting," in *International Society for Music Information Retrieval Conference (ISMIR)*, 2020.

[11] R. Louie, J. H. Engel, and C. A. Huang, "Expressive communication: A common framework for evaluating developments in generative models and steering interfaces," in *ACM Intelligent User Interfaces Conference (IUI)*, 2022.

[12] H. Liu, Q. Tian, Y. Yuan, X. Liu, X. Mei, Q. Kong, Y. Wang, W. Wang, Y. Wang, and M. D. Plumbley, "AudioLDM 2: Learning holistic audio generation with self-supervised pretraining," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2871–2883, 2024.

[13] P.-Y. Huang, H. Xu, J. Li, A. Baevski, M. Auli, W. Galuba, F. Metze, and C. Feichtenhofer, "Masked autoencoders that listen," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[14] B. Han, J. Dai, X. Song, W. Hao, X. He, D. Guo, J. Chen, Y. Wang, and Y. Qian, "InstructME: An instruction guided music edit and remix framework with latent diffusion models," *arXiv preprint arXiv:2308.14360*, 2023.

[15] A. S. Hussain, S. Liu, C. Sun, and Y. Shan, "M^{2}UGen: Multi-modal music understanding and generation with the power of large language models," *arXiv preprint arXiv:2311.11255*, 2023.

[16] Y. Zhang, Y. Ikemiya, G. Xia, N. Murata, M. Martínez, W.-H. Liao, Y. Mitsufuji, and S. Dixon, "MusicMagus: Zero-shot text-to-music editing via diffusion models," in *Proceedings of International Joint Conferences on Artificial Intelligence (IJCAI)*, 2024.

[17] M. Plitsis, T. Kouzelis, G. Paraskevopoulos, V. Katsouros, and Y. Panagakis, "Investigating personalization methods in text to music generation," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 1081–1085.

[18] H. Ye, J. Zhang, S. Liu, X. Han, and W. Yang, "IP-Adapter: Text compatible image prompt adapter for text-to-image diffusion models," *arXiv preprint arXiv:2308.06721*, 2023.

[19] D. Ellis, "Chroma feature analysis and synthesis," *Resources of laboratory for the recognition and organization of speech and Audio-LabROSA*, 2007.

[20] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[21] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *International Conference on Learning Representations (ICLR)*, 2021.

[22] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF*

Conference on Computer Vision and Pattern Recognition (CVPR), 2022.

[23] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[24] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma *et al.*, "Scaling instruction-finetuned language models," *Journal of Machine Learning Research (JMLR)*, 2024.

[25] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[26] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *Open AI Blog*, 2019.

[27] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical Image Computing and Computer-Assisted Intervention*, 2015.

[28] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *arXiv preprint arXiv:2207.12598*, 2022.

[29] R. Gal, M. Arar, Y. Atzmon, A. H. Bermano, G. Chechik, and D. Cohen-Or, "Encoder-based domain tuning for fast personalization of text-to-image models," *ACM Transactions on Graphics (TOG)*, 2023.

[30] N. Kumari, B. Zhang, R. Zhang, E. Shechtman, and J.-Y. Zhu, "Multi-concept customization of text-to-image diffusion," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 1931–1941.

[31] H. concept library, "SD-Dreambooth-Library," https://huggingface.co/sd-dreambooth-library, 2024, [Online; accessed 10-April-2024].

[32] Stable Diffusion Art, "How does negative prompt work?" 2024, [Online; accessed 10-April-2024]. [Online]. Available: https://stable-diffusion-art.com/how-negative-prompt-work/

[33] G. Sanchez, H. Fan, A. Spangher, E. Levi, P. S. Ammanamanchi, and S. Biderman, "Stay on topic with classifier-free guidance," *arXiv preprint arXiv:2306.17806*, 2023.

[34] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio Set: An ontology and human-labeled dataset for audio events," in *IEEE international conference on acoustics, speech and signal processing (ICASSP)*, 2017.

[35] "AI Hub Dataset," https://www.aihub.or.kr/aihubdata/data/view.do?currMenu=115&topMenu=100&dataSetSn=71470.

[36] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi, "Fréchet Audio Distance: A metric for evaluating music enhancement algorithms," *arXiv preprint arXiv:1812.08466*, 2018.

[37] C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, and S. Ermon, "SDEdit: Guided image synthesis and editing with stochastic differential equations," in *International Conference on Learning Representations (ICLR)*, 2022.

[38] B. McFee, C. Raffel, D. Liang, D. P. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in python." in *SciPy*, 2015.

[39] S. Hershey, S. Chaudhuri, D. P. Ellis, J. F. Gemmeke, A. Jansen, R. C. Moore, M. Plakal, D. Platt, R. A. Saurous, B. Seybold *et al.*, "CNN architectures for large-scale audio classification," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017.